Secure File Storage System with AES-256

1. Introduction

This project implements a secure file storage system using AES-based encryption to protect user files. It ensures confidentiality by encrypting files before storage and supports safe retrieval through decryption. The system demonstrates practical cryptographic techniques used in cybersecurity.

2. Abstract

The Secure File Storage System is a Python-based application that uses the cryptography library (Fernet) to perform AES encryption and decryption of files. Users can securely upload, encrypt, store, and retrieve files. The system also generates SHA-256 hashes to verify file integrity and prevent tampering. A CLI-based interface provides easy interaction for users.

3. Tools Used

1 Python 3.x

2 Cryptography Library (Fernet/AES)

3 Hashlib (SHA-256)

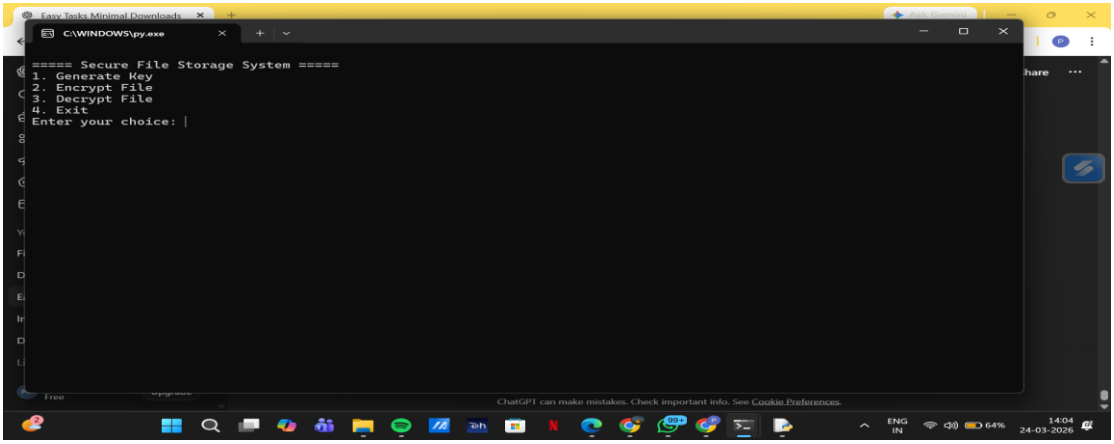
4 Command Line Interface (CLI) / IDLE

4. Steps Involved

Key generation is performed and stored securely. Files are read in binary format and encrypted using AES, then saved with a .enc extension. Metadata including file name, timestamp, and hash is stored. During retrieval, the system verifies file integrity using SHA-256 before decrypting and restoring the original file.

5. Conclusion

The project successfully demonstrates secure file storage using AES encryption and integrity verification. It highlights real-world applications of cryptography, ensuring confidentiality and protection against tampering. This system provides a practical understanding of secure data handling in cybersecurity.



C:\WINDOWS\py.exe

```
===== Secure File Storage System =====
1. Generate Key
2. Encrypt File
3. Decrypt File
4. Exit
Enter your choice: 1
✅ Key generated and saved in keys/secret.key
```

```
===== Secure File Storage System =====
1. Generate Key
2. Encrypt File
3. Decrypt File
4. Exit
Enter your choice: 2
Enter file path to encrypt: C:/Users/prakr/OneDrive/Desktop/test.txt
✅ File encrypted successfully!
📁 Saved as: encrypted_files\test.txt.enc

===== Secure File Storage System =====
1. Generate Key
2. Encrypt File
3. Decrypt File
4. Exit
Enter your choice: 3
Enter encrypted file path: encrypted_files/test.txt.enc
🔍 File hash: 432784fd68c255bf1696248bca4e0502c8fae97944a6b553dcde134c86b96f0a
✅ File decrypted successfully!
📁 Saved as: decrypted_test.txt

===== Secure File Storage System =====
1. Generate Key
2. Encrypt File
3. Decrypt File
4. Exit
Enter your choice:
```

secure_file_storage

Start OneDrive › Prakruthi - Personal › Desktop › secure_file_storage ›

Search secure_file_storage

New · Sort · View · Details

Prakruthi - Personal

Desktop
Downloads
Documents
Pictures
Music
Videos
Screenshots
p doc
resumes
secure_file_storage

This PC
OS (C:)
Network

Name	Status	Date modified	Type	Size
encrypted_files		24-03-2026 14:13	File folder	
keys		24-03-2026 14:08	File folder	
decrypted_test.txt		24-03-2026 14:15	Text Document	1 KB
main.py		24-03-2026 14:00	Python File	4 KB
metadata.json		24-03-2026 14:13	JSON File	1 KB

5 items

ENG IN 52% 14:17 24-03-2026